$ ls ~/prompts

The prompt library I actually use.

The prompts that run parts of my business, plus the best patterns from operators I follow, rewritten for real work: ops, sales, content, coding, and research.

33 prompts · 5 categories · v1.0 · September 2026
joestechsolutions.com/prompt-library · joe@joestechsolutions.com

How to use these

Paste a prompt, replace the [brackets], and give the model real material: a real email thread, a real diff, a real competitor. Vague in, vague out. Most of these work in Claude, ChatGPT, or a local model through Ollama; the coding ones assume an agent that can read your repo (Claude Code, Cursor).

- **Role, task, context, constraints, format.** Every good prompt has the same skeleton: who the model is, what you want, what it needs to know, what it must not do, and what the output should look like. Half the prompts here are just that skeleton filled in well.
- **Interview me first.** The single most shared move across every creator we looked at: make the model ask you questions before it answers. Generic input produces generic output; five questions fix that.
- **Show it what good looks like.** One real example beats twenty rules. Paste a sample you love and say match it, or paste one you hate and say never like this.
- **Definition of done, then loop.** Tell the model how it will know it is finished, in checkable terms: a passing test, a number, a screenshot. Then let it iterate until it gets there instead of stopping at good enough.
- **Argue against yourself.** Ask for blind spots, a pre-mortem, or the strongest case against your own plan. Models are eager to agree with you unless you tell them not to.
- **Plan first, then small steps.** For anything with side effects (code, money, customers) ask for the plan and the list of changes before anything moves. Fewer rules per prompt, one scoped step at a time.
- **Sources or it did not happen.** For research, require a URL for every claim and an explicit list of assumptions. Unverified claims get marked, not dressed up.
- **Give it memory.** Store your business profile, voice, and standing rules once, in a file the model reads every time. Then prompts get shorter and answers get better.
- **Humanize the output.** Short sentences, plain words, no em dashes, no hype vocabulary. If a sentence sounds like a press release, cut it.

01 Ops

Daily briefings, meeting prep, and task triage: the prompts that run a morning.

The morning briefing

Use it when: First thing, before you open anything else. Feed it your calendar, inbox and task list (export, paste, or let an agent read them).

```
You are my chief of staff. Here is today's calendar, my inbox from the last 24
hours, and my open task list:

[PASTE CALENDAR]
[PASTE INBOX SUMMARY OR THREADS]
[PASTE TASK LIST]

Give me a briefing I can read in 90 seconds:
1. The one thing I should do first, and why.
2. Anything blocking someone else on me (name the person and what they need).
3. Anything overdue, oldest first.
4. Meetings today: for each, one line on what I need to have ready.
5. Things I can safely ignore until tomorrow.

Be blunt. No pleasantries, no restating my calendar back to me.
```

Make it yours: Run it on a schedule with an agent and it becomes the daily briefing that opens your day. Add a line about your priorities for the week and it will re-rank around them.

Meeting notes → actions → recap email

Use it when: Right after any meeting, with the transcript, your notes, or a voice memo transcription.

```
Here are the notes from a meeting I just had:

[PASTE TRANSCRIPT OR NOTES]

Do four things, in this order:
1. Topics, decisions, and anything surprising, as bullets. Decisions get a bold
"DECIDED:" prefix.
2. Action items as a table: owner, task, deadline. If a deadline was not said,
write "unset". Do not invent one.
3. Open questions nobody answered.
4. A recap email to the attendees, under 150 words, that I can send as-is. Plain,
no "I hope this finds you well."

If anything is ambiguous (who owns what, what was actually decided), ask me
before writing the email.
```

Make it yours: Ask for the recap in the meeting's own language: if they said "ship", do not let the model write "deploy to production".

Pattern credit: Sabrina Ramonov's meeting-notes prompt; Anthropic's "action items → tickets" pattern

What will they ask me?

Use it when: The night before a client call, a pitch, a review, or any meeting where you are the one being questioned.

```
I have a meeting tomorrow. Context:

Who: [WHO, THEIR ROLE, WHAT THEY CARE ABOUT]
About: [PASTE THE DOC, PROPOSAL, OR STATUS I AM PRESENTING]
What I want out of it: [DECISION / APPROVAL / MONEY / TIME]

1. Predict the 10 questions I am most likely to get, hardest first. Include the
awkward ones (price, timeline, why not the cheaper option, what if it breaks).
2. For each, a two-sentence answer in my voice, plus the one number or fact I
should have ready.
3. The three questions I should ask them, so I leave with what I need.

Do not soften the hard questions. If my material has a hole they will find, say
so.
```

Make it yours: Paste the last email thread with them too. The questions get far more specific.

Pattern credit: Sabrina Ramonov's /QUESTIONPREDICTOR; Lenny's Newsletter's meeting-prep prompts

Impact/effort triage of the task list

Use it when: When the list is longer than the week and everything feels urgent.

```
You are a blunt operations lead. Here is my task list:

[PASTE TASKS, ONE PER LINE, WITH ANY DEADLINES]

For each task estimate impact (what it changes for revenue, customers, or risk)
and effort (hours, honestly). Then:
- Plot them in four groups: do now (high impact, low effort), schedule
(high/high), delegate or automate (low/low), delete (low impact, high effort).
- For the "delete" group, say why in one line each. I want permission to drop
them.
- Give me a Monday-to-Friday order for the top five, with the reason each one
goes where it goes.

If a task is really three tasks, split it and say so.
```

Make it yours: Run it every Monday with an agent and paste last week's list in too. It will notice what keeps slipping.

Pattern credit: Ruben Hassid's impact/effort framework prompt

Interview me first

Use it when: Before any task that matters: a proposal, a plan, a piece of writing, a decision. Put this in front of the real request.

```
Before you do anything, interview me.

The task: [DESCRIBE WHAT YOU WANT]

Ask me the five questions whose answers would most change your output, about
audience, constraints, what I have already tried, what "good" looks like, and
what I am afraid of getting wrong. One at a time or all at once, your call. Do
not start on the task until you are at least 90 percent confident you understand
it. Then restate the task in two sentences and begin.
```

Make it yours: Works in any tool. In Claude Code, this is what plan mode is for; in Claude Cowork, ask for the question form directly.

Pattern credit: The most shared pattern in the research: Sabrina Ramonov, Ruben Hassid, RoboNuggets, Greg Isenberg, Nate Herk, and Anthropic's spec-by-interview all use it

The overdue-invoice ladder

Use it when: Invoices past due. Feed it the thread so it does not nag someone who already replied.

```
Write the next collection email for this invoice.

Client: [NAME, COMPANY]
Invoice: [NUMBER, AMOUNT, DUE DATE]
Days overdue: [N]
Our thread so far: [PASTE THE EMAIL THREAD, OR "none"]

Rules:
- Level 1 (1-7 days): friendly, assume they forgot, attach the link.
- Level 2 (8-14 days): direct, ask if there is a problem with the work.
- Level 3 (15-28 days): firm, name the date service pauses.
- Level 4 (29+ days): final notice, plain and short, no apology.
- Pick the level from the days overdue, but read the thread first: if they
replied with a reason or a date, respond to that instead of escalating.
- Under 120 words. My voice: direct and warm, never passive-aggressive.
```

Make it yours: Put the four levels in your agent's memory once and it becomes a weekly job: it checks the ledger, reads the threads, and drafts what is due.

Pattern credit: Nick Saraev's invoice collection workflow

The Friday review

Use it when: End of week, with whatever you have: notes, a project folder, closed tickets, the calendar, the bank feed.

```
Read everything I give you and write my Friday review:

[PASTE OR ATTACH: the week's notes, task list with done/not done, calendar,
anything else]

Format, fixed:
- Shipped: what actually got done. Facts, not feelings.
- Slipped: what did not, and the real reason (name it: scope, waiting on someone,
avoidance).
- Money: anything invoiced, paid, or overdue this week.
- Next week: the three things that matter most, and the one thing to stop doing.
- One question for me that you could not answer from the material.

Under 300 words. No praise. If the week was bad, the review should read that way.
```

Make it yours: This is the executive-review shape Joe runs on schedule in Hermes. The fixed format is the point: you learn to skim it in 30 seconds.

Pattern credit: Ruben Hassid's scheduled executive briefing; the weekly-review cadence in Nate Herk's AIS-OS

02 Sales

Outreach, follow-ups, and lead qualification that don't sound like a robot wrote them.

The follow-up that references what they actually said

Use it when: Any time a thread has gone quiet after a call, a demo, or a proposal.

```
Draft a follow-up to this email thread:

[PASTE THREAD]

Rules:
- Reference something specific they said or asked for. Quote their words if it
helps.
- Under 100 words.
- One concrete next step with a date or a yes/no question. Not "let me know your
thoughts."
- No "just checking in", no "circling back", no apologies for following up.
- Match their tone: if they write short, write short.

Write it in my voice: direct, warm, zero filler.
```

Make it yours: Paste two or three of your own past emails first and say "write like this". The voice match gets dramatically better.

The non-obvious icebreaker

Use it when: Cold outreach. Give it the prospect's website pages, not just the homepage.

```
You are researching a company so I can open an email with something real.

Their website (pasted or crawled): [PASTE THE ABOUT PAGE, A SERVICES PAGE, AND
ONE RECENT POST OR NEWS ITEM]
My offer, in one line: [OFFER]

Write a two-sentence opener that proves I looked. Rules:
- Pick something small and non-obvious: a detail from their about page, a choice
they made, a recent change. Never "love what you're doing" or a compliment about
their homepage.
- Shorten the company name the way a friend would.
- Plain, spartan tone. No exclamation marks.
- Then one sentence that bridges to why I am writing, without pitching yet.

Give me three options and mark the one you would send.
```

Make it yours: Keep the personalization in the opener only; template the rest of the email. One real detail beats a whole personalized page.

Pattern credit: Nick Saraev's website-scrape icebreaker

Pre-call brief

Use it when: Fifteen minutes before a sales or discovery call, with whatever you know about them.

```
Build me a pre-call brief.

Company: [NAME, WEBSITE, INDUSTRY, SIZE]
Contact: [NAME, TITLE]
What I know so far: [NOTES, PAST EMAILS, HOW THEY FOUND ME]
What I sell: [ONE LINE]

Sections, each 3-5 bullets, no fluff:
- Company in brief: what they do and who they sell to.
- Likely pain points for a business like theirs (label guesses as guesses).
- Where my offer fits, and where it probably does not.
- Discovery questions on need, budget, timeline, and who decides.
- Anything recent (news, hiring, a new location) worth mentioning.

Bold the one thing I should not forget to ask.
```

Make it yours: If the model can browse, add "check their site and recent news before answering" and it stops guessing at the basics.

Pattern credit: Sabrina Ramonov's B2B sales call brief

Inquiry reply plus two follow-ups

Use it when: A new inbound lead by email, form, or DM. Local-service businesses especially.

```
A prospect just reached out. Their message:

[PASTE THEIR MESSAGE]

About my business: [SERVICES, TYPICAL PRICING OR HOW I QUOTE, AVAILABILITY, WHAT
MAKES ME DIFFERENT]

Write three emails:
1. The reply, today: acknowledge exactly what they asked, answer what I can, say
how quoting or booking works, and end with one clear next step.
2. A follow-up for day 3 if they go quiet: restate the value in one line, remove
the most likely hesitation (price, timing, trust), and offer an easy way to say
yes or no.
3. A final note for day 10: respectful, one sentence of value, the door is open,
no pressure.

Under 120 words each. Sound like a person who wants the work, not a funnel.
```

Make it yours: Paste two of your own past replies first and say "this is my voice". The drafts stop sounding like a template.

Pattern credit: Sabrina Ramonov's customer-responses prompt

Give before you ask

Use it when: Outreach to prospects who ignore normal cold email. Higher effort, higher reply rate.

```
I want to send [PROSPECT] something useful before asking for anything.

What I know about them: [WEBSITE PAGES, RECENT POSTS, WHAT THEY SELL]
What I am good at: [YOUR SKILL OR SERVICE]

Write a finished, specific piece of work they could use today, for example a
teardown of their booking flow with three fixes, a rewritten version of one weak
page, or a short plan for a problem their site shows. Make it real enough that
they could act on it without hiring me.

Then write the cover note: two sentences. "I put this together for you, hope it
helps," and nothing else. No pitch, no call request.
```

Make it yours: This decays as more people do it. It works because the piece is genuinely good, so do not let the model phone it in: review it like it is client work.

Pattern credit: Nick Saraev's give-before-you-ask outreach

Proposal from discovery notes

Use it when: After the call, with your notes. The questions-first line is what keeps it from inventing scope.

```
Here are my notes from a discovery call:

[PASTE NOTES]

Before writing anything, ask me the questions you need answered to scope this
honestly: what is in, what is out, what depends on them, what could go wrong.

Then write a proposal with these sections, short:
- What they told me the problem is (in their words where I have them).
- What I will build or do, as a numbered list with a definition of done for each
item.
- What is explicitly out of scope.
- Timeline with the dependencies on their side called out.
- Price: leave a [PRICE] placeholder. Never invent a number.
- What happens next: one sentence.

Plain English. A reader with five minutes should know exactly what they are
buying.
```

Make it yours: Joe's rule: scope and price before any code. The out-of-scope section saves more relationships than the rest of the document combined.

Pattern credit: Ruben Hassid's discovery-notes-to-proposal workflow; Sabrina Ramonov's proposal skeleton

03 Content

Blog outlines, social posts, and newsletters in your voice, not generic AI filler.

Build notes → blog outline

Use it when: You shipped something and have messy notes, a changelog, or a commit log.

```
Turn these build notes into a blog post outline:

[PASTE NOTES / CHANGELOG / COMMITS]

Structure:
- Lead with the problem the reader has, not with what I built.
- Show the real numbers where I have them (time saved, cost, error rates). If a number is missing, put [NUMBER?] so I can fill it in. Never invent one.
- One section on what went wrong or what I'd do differently. Readers trust that more than the wins.
- End with what the reader can copy from this, in 3 bullets.

No hype words (game-changing, revolutionary, seamless, unlock). Plain English.
Give me headings plus 1-2 sentences per section, not the full post yet.
```

Make it yours: Ask for the full draft only after you've edited the outline. Fixing structure is cheap; fixing 1,200 words of the wrong structure is not.

Humanize this

Use it when: Any AI-assisted draft before it goes out. Run it last.

```
Rewrite this so it reads like a person wrote it:

[PASTE DRAFT]

Rules:
- Short sentences. Active voice. Talk to the reader as "you".
- Concrete examples and numbers over adjectives.
- No em dashes, no semicolons, no markdown, no hashtags, no rhetorical questions.
- Remove these words entirely: unlock, leverage, seamless, robust, delve,
elevate, game-changing, revolutionize, empower, journey, landscape, tapestry,
testament, harness, cutting-edge, navigate, realm, embark, transformative,
holistic.
- Keep my meaning and my facts exactly. Do not add anything.

Return only the rewrite.
```

Make it yours: Add three sentences you actually wrote and say "keep this rhythm". Banned-word lists remove the AI smell; a sample gives it a voice to replace it with.

Pattern credit: Sabrina Ramonov's humanize prompt; Ruben Hassid's banned-word list

Match this, never that

Use it when: Any writing task where you already know what good sounds like.

```
I want you to write [WHAT: a post, an email, a page, a caption] about [TOPIC].

Here is a sample I think is excellent. Match its rhythm, length, tone, and
structure:
[PASTE THE GOOD SAMPLE]

Here is a sample I never want to sound like. Avoid everything about it:
[PASTE THE BAD SAMPLE]

Do not copy phrases from either. Learn the shape, then write mine. Give me two
versions: one that plays it safe, one that takes a swing.
```

Make it yours: The negative example does more than the positive one. Most people only paste what they like.

Pattern credit: Sabrina Ramonov's "here is what great looks like"; Ruben Hassid's "never write like this"

The post grader

Use it when: Before publishing anything on LinkedIn, X, or a newsletter. Also works on landing-page copy.

```
Grade this post against the rules below, then fix it.

Post: [PASTE POST]
Audience: [WHO]
Goal: [REPLIES / CLICKS / DMS / TRUST]

Rules to grade against (1-10 each): the first line makes someone stop scrolling;
one idea, not three; specific over general; a real example or number; reads well
on a phone; ends with something to do or think.

Output:
1. The scores, one line each, with the single reason for anything under 8.
2. The three highest-impact fixes, in order.
3. The rewritten post with those fixes applied.
4. The new scores.

If it is already above 8 everywhere, say so and change nothing.
```

Make it yours: Loop it: paste the rewrite back in until everything is 8 or higher. Two rounds is usually enough; five means the idea is the problem.

Pattern credit: Sabrina Ramonov's post-grader skill

Ten angles, with sources

Use it when: Planning a newsletter or a month of posts. Best with a model that can search.

```
Act as my research lead. I write for [AUDIENCE] about [TOPICS].

Find me ten possible pieces for the next month. For each: the topic, the angle
(what I would say that others are not), the source or event it hangs on (URL),
and one line on why my audience would care.

Prefer: counterintuitive findings, things that launched or changed this month, a
hidden feature or trick, a contrarian take backed by data. Skip anything that has
already been covered to death.

Rank them. Mark the two you would write first and say why.
```

Make it yours: Paste your last five posts or issues so it stops proposing what you already wrote.

Pattern credit: Ruben Hassid's research-lead prompt

Repurpose one piece into three

Use it when: After a blog post, a case study, or a long email you are proud of.

```
Here is a piece I wrote:

[PASTE THE PIECE]

Turn it into three things, each standing on its own:
1. A LinkedIn post, under 200 words, opening with the most surprising line in the piece, no hashtags.
2. An X thread of 5-7 posts, each under 240 characters, the first one a hook that works without the rest.
3. A 45-second talking-head script, spoken language, one idea, ending with a question.

Keep my facts and numbers exactly. Do not add claims. Match the voice of the original, not "social media voice".
```

Make it yours: Run the post grader on each output. Repurposed content is where the AI smell creeps back in.

Pattern credit: Sabrina Ramonov's repurpose skill; Greg Isenberg's model-rivalry trick works well here: give the same piece to two models and keep the best parts

04 Coding

Code review, debugging, and working with coding agents like Claude Code and Cursor.

Review this diff for bugs only

Use it when: Before you merge anything an AI agent (or you at 11pm) wrote.

```
Review this diff for bugs only. No style nits, no naming opinions, no "consider
adding tests" unless a missing test would have caught a bug you found.

[PASTE DIFF or point the agent at the branch]

For each issue:
- The exact failure case: what input or state triggers it, and what goes wrong.
- Severity: would ship broken / would ship wrong data / edge case.
- The smallest fix.

If you find nothing, say "no bugs found" and list the two places you looked
hardest. Do not invent problems to look thorough.
```

Make it yours: Run it twice with different models if it's a payment path or anything touching user data. Disagreement between the two reviews is where the real bugs hide.

Spec by interview

Use it when: Before building any feature bigger than an afternoon. Works in Claude Code, Cursor, or a chat.

```
I want to build [FEATURE, ONE SENTENCE].

Interview me about it before writing anything: the user flow, edge cases, what
happens on failure, data that has to survive, what is out of scope, and the
tradeoffs you see. Ask until you have covered everything, then write the spec to
SPEC.md with these sections: goal, user flow, data, edge cases, out of scope,
definition of done (testable), open questions.

Do not write code. Do not pick a library yet.
```

Make it yours: Keep SPEC.md in the repo. Every later prompt starts with "read SPEC.md", and the agent stops drifting.

Pattern credit: Anthropic's Claude Code prompt library (spec by interview)

Plan first, touch nothing

Use it when: Refactors, migrations, anything that changes more than one file.

```
Plan how to [GOAL: refactor X to Y / migrate from A to B / bring metric M under N].

List every file you would change and what changes in each, in the order you would do them. Call out anything risky: shared state, migrations, places where behavior could change for users. Estimate how you would verify each step.

Do not change anything yet. When I approve the plan, we go one step at a time, and you run the verification after each step before moving on.
```

Make it yours: Approve the plan, then say "step 1 only". Agents that get the whole plan at once tend to do all of it at once.

Pattern credit: Anthropic's Claude Code prompt library (plan-first refactors); Greg Isenberg and Ross Mike's plan-mode workflow

Definition of done, with a check and a limit

Use it when: Any agent task you want to leave running. Fuzzy goals are how agents burn an afternoon.

```
Goal: [ONE SENTENCE. No fuzzy words like clean, polished, better, robust.]

Definition of done: [SOMETHING CHECKABLE: the test passes, the page loads under 1s, the CSV has 500 rows with no blanks]
Out of scope: [AT LEAST TWO THINGS YOU MUST NOT TOUCH]
Check: after every change, run [COMMAND / TEST / SCREENSHOT] and report the result before continuing.
Limit: stop after [25 / 50 / 100] turns and report where you are, even if not done.

Do not stop early because it is "good enough". Do not stop late because you are close. Meet the definition of done or hit the limit, then report.
```

Make it yours: The out-of-scope lines matter as much as the goal. "Do not touch the payment code" has saved more than one weekend.

Pattern credit: RoboNuggets' /goal plan; Sabrina Ramonov's "don't stop until, verified by"

Investigate the incident

Use it when: Something is wrong in production and you do not know why yet.

```
Users are seeing [SYMPTOM] on [WHERE], starting around [WHEN].

Investigate before touching anything: check the logs for that window, the last
three deploys, and any config or environment changes. Then tell me:
1. The most likely cause, with the evidence that points to it.
2. Two other possibilities and what would rule them out.
3. The smallest safe fix, and the smallest safe rollback if the fix is wrong.

Do not fix it yet. Do not guess at a cause you cannot show me evidence for.
```

Make it yours: Ask for the evidence every time. An agent that says "probably the cache" without a log line is guessing, and so are you if you accept it.

Pattern credit: Anthropic's Claude Code prompt library (debugging and incidents)

The four rules for your coding agent

Use it when: Paste into CLAUDE.md, .cursorrules, or whatever file your agent reads first. Once.

```
Working rules for this repo:

1. Before writing code, state your assumptions and the tradeoff you are choosing.
If a requirement is ambiguous, ask; do not pick silently.
2. Write the minimum code that solves the request. No extra abstractions, no
"while I'm here" changes, no new files unless the task needs them.
3. Edit surgically. Match the patterns already in this codebase, even if you
would have done it differently.
4. Turn every task into a checkable success criterion before you start, verify it
when you finish, and say what you verified. Never make a test pass by special-
casing the test.
```

Make it yours: Add one rule at a time when the agent repeats a mistake: "you keep doing X, add a rule so it stops". Ten rules is a lot; four good ones is plenty.

Pattern credit: The Karpathy-style CLAUDE.md popularised by RoboNuggets; Anthropic's agentic coding guidance

Steer, don't restart

Use it when: Mid-session, when the agent overreaches or repeats itself. Three short prompts worth keeping close.

```
When it did too much:
"That is too much. Keep only the changes to [SCOPE] and undo everything else.
Show me the diff of what remains."

When it keeps making the same mistake:
"You keep [MISTAKE]. Add a rule to CLAUDE.md that stops this, in one sentence,
then continue."

When the session gets long:
"Summarize what we did, what is verified, and what is still open, in a form I can
paste into a fresh session. Then suggest what belongs in CLAUDE.md."
```

Make it yours: Restarting a session throws away context you paid for. Steering keeps it. But once you are past about half the context window, the summary-and-restart is the better move.

Pattern credit: Anthropic's Claude Code prompt library (steering and memory); Greg Isenberg's context-budget rule

05 Research

Competitor scans, tool comparisons, and decision briefs that come back with sources, not vibes.

Compare three tools, with sources

Use it when: You're about to pay for software and the marketing pages all say the same thing.

```
Compare [TOOL A], [TOOL B] and [TOOL C] for this use case:

[DESCRIBE YOUR USE CASE, TEAM SIZE, BUDGET, MUST-HAVES]

Rules:
- Cite a source (URL) for every factual claim: pricing, limits, integrations. If
you can't source it, mark it "unverified".
- Pricing in real numbers at my usage level, not "starts at".
- Deal-breakers first: anything that fails a must-have ends that tool's
evaluation.
- Then a table: cost/month, setup effort, lock-in risk, the one thing each does
best.
- Finish with what you'd pick and why, in three sentences. Then the strongest
argument against your own pick.
```

Make it yours: Add "assume I will run this for two years". The answer changes once migration cost and lock-in are on the table.

Mine the complaints

Use it when: Before building a product, an offer, or a page. Best with a model that can search, or paste the threads yourself.

```
Find what people are actually complaining about in [CATEGORY / PRODUCT / TYPE OF
BUSINESS], from Reddit, app-store reviews, and forums, since [DATE].

For each complaint theme: a short label, how often it shows up, two quoted
snippets with links, and what the person was trying to do when it failed.

Group them. Rank by frequency. Then tell me the two themes nobody seems to be
solving well, and why you think that is.

Do not paraphrase into marketing language. Keep the quotes ugly.
```

Make it yours: This is where offers come from. The ugly quotes go straight into your page copy, in their words.

Pattern credit: Ruben Hassid's Reddit pain-point prompt; the same move from Sabrina Ramonov and Greg Isenberg

Find my blind spots

Use it when: Any plan you are attached to. Especially the ones you are excited about.

```
Here is my plan:

[PASTE PLAN, PROPOSAL, OR DECISION]

Argue against it. Specifically:
1. The three assumptions I am making without evidence, ranked by how much the
plan depends on them.
2. Who benefits if this fails, and who would quietly not cooperate.
3. The strongest version of the case against, as if you were paid to make it.
4. What I would need to see in the first two weeks to know it is working, and
what would tell me to stop.

No flattery. No "great plan, but". Start with the weakest point.
```

Make it yours: Prefix a second run with "first principles": rebuild the plan from what is true rather than what is customary. Different answers, both useful.

Pattern credit: Sabrina Ramonov's blind-spots and REDTEAM prompts; Lenny's Newsletter's pre-mortem

The pre-mortem

Use it when: Before launching a feature, a service, or a campaign.

```
It is six months from now and [PROJECT] has failed.

Write the post-mortem:
- Five reasons it failed, most likely first, each with the early warning sign we
ignored.
- Five unintended consequences it caused, even where it partly worked.
- The one decision we would most want back.

Then flip it: for each reason, the cheapest thing we could do this week to reduce
the chance of it happening.
```

Make it yours: Do this with the people who will actually run the thing, and let the model go first. It says the awkward parts so nobody has to.

Pattern credit: Lenny's Newsletter's pre-mortem prompt

Stress test, then one verdict

Use it when: Choosing between ideas, offers, or directions when you could argue for any of them.

```
I am choosing between these options:

[LIST 2-5 OPTIONS, ONE LINE EACH]

My situation: [SKILLS, TIME, MONEY, EXISTING CUSTOMERS OR AUDIENCE]

For each option:
- The core assumption most likely to be wrong.
- What the first 90 days actually involve, day to day. Be specific enough that I
can picture it.
- The reason people quit this one.

Then score each on: paying customers within 30 days, still appealing in two
years, uses what I already have, recoverable if it fails. Score 1-5 each, show
the table, and pick one winner. Not "it depends". One.
```

Make it yours: Force the single verdict. The model will hedge if you let it, and hedged advice is the same as none.

Pattern credit: Sabrina Ramonov's stress-test-to-verdict prompts

Competitor scan

Use it when: Once a quarter, or when someone new shows up in your market.

```
Analyse these competitors against my business.

Me: [WEBSITE, WHAT I SELL, WHO TO]
Them: [2-4 COMPETITOR WEBSITES, PLUS THEIR GOOGLE PROFILE OR SOCIAL LINKS IF
RELEVANT]

For each competitor: what they sell and to whom; how they position (price, speed,
quality, niche); what they publish and how often; what customers praise and
complain about in reviews (quote two); what they do that I do not.

Then: the three gaps in the market none of us is filling, the one thing I should
copy, and the one thing I should deliberately not compete on.

Cite the page or review for each claim. Mark anything you inferred rather than
read.
```

Make it yours: Repeat with the same competitors next quarter and ask what changed. The diff is more useful than the snapshot.

Pattern credit: Sabrina Ramonov's competitor-analysis prompt

Five whys

Use it when: When the same problem keeps coming back: a missed deadline, a churned client, a bug that reappears.

```
Act as a root-cause analyst. The problem:

[DESCRIBE WHAT HAPPENED, WITH DATES AND WHO WAS INVOLVED]

Ask "why did that happen?" and answer it. Then ask why of that answer. Do it five
times, or until the answer is something we can change rather than something that
just is. Show the chain.

Then give me: the root cause in one sentence, the fix that addresses the root
cause (not the symptom), and how we would know in 30 days that it worked.

If you hit a point where you would be guessing, stop and ask me.
```

Make it yours: The fifth why is usually about a person, a process, or a missing check. Fix that one, not the first.

Pattern credit: Ruben Hassid's Five Whys framework prompt

~ Credits and sources

Patterns are ideas, and good ones get shared. These are the operators whose public prompts and techniques shaped this library. Every prompt here was rewritten for a small-business context; nothing is copied.

- **Sabrina Ramonov**: public prompt repo (meeting notes, sales call brief, customer responses, competitor analysis), the humanize prompt, and her Claude prompt lists https://github.com/SabrinaRamonov/prompts
- **Sabrina Ramonov**: the six Claude prompts (clarifying questions, blind spots, match-the-sample, definition of done) https://www.sabrina.dev/p/the-6-claude-prompts-to-get-you-ahead
- **RoboNuggets**: the gauntlet loop, the /goal plan (definition of done, out of scope, check, turn limit), the Karpathy-style CLAUDE.md https://www.robonuggets.blog/p/the-future-of-ai-agents-just-arrived
- **Greg Isenberg**: model rivalry, plan mode and feature-based development with Claude Code, research-before-you-prompt https://x.com/gregisenberg/status/1918661279641723158
- **Ruben Hassid**: Prompt Maker, the eight thinking-framework prompts (Five Whys, impact/effort, JTBD), search prompts with sources, the Cowork workflows https://ruben.substack.com/p/prompt-maker
- **Nick Saraev**: the website-scrape icebreaker, the cold-email formula, give-before-you-ask outreach, the invoice escalation ladder, the prompt scaffold https://www.youtube.com/watch?v=CxbHw93oWP0
- **Nate Herk**: AIS-OS (/onboard, /audit, /level-up) and the Role / Objective / Context / Instructions / Examples / Notes framework https://github.com/nateherkai/AIS-OS
- **Lenny's Newsletter**: PM prompts (stakeholder-persona review, pre-mortem, hidden assumptions) and five proven prompt techniques https://www.lennysnewsletter.com/p/how-to-use-chatgpt-in-your-pm-work
- **Matt Wolfe**: reflection loops, tree-of-thought, the humanize shortcut https://youtu.be/aO-HoPDUges
- **Liam Ottley**: the free-system outreach playbook https://x.com/liamottley_/status/1701195864583393330
- **Anthropic**: the Claude Code prompt library (spec by interview, plan-first refactors, incident investigation, steering) and prompting best practices https://code.claude.com/docs/en/prompt-library
- **The Rundown AI and Ben's Bites**: the task / context / examples / persona / format / tone basics https://www.therundown.ai/ai-tools-course-chapter-1-chatgpt

---

Joe's Tech Solutions: custom software, AI agents, automation, and private AI for small businesses. New prompts land at joestechsolutions.com/prompt-library. Questions: joe@joestechsolutions.com.